RAPPORT DE TRAVAUX PRATIQUES

TP 1 : Git & Node.js

Module	Génie Logiciel
Encadrant	Pr. BE. ELBAGHAZAOUI
Réalisé par	Ikram El-MAZINI
Année universitaire	2025 / 2026

1. Introduction

Ce travail pratique a pour objectif de se familiariser avec Git, un système de contrôle de version distribué, ainsi qu'avec Node.js, un environnement d'exécution JavaScript côté serveur. À travers plusieurs exercices progressifs, nous avons appris à initialiser un dépôt, gérer des branches, résoudre des conflits de fusion, effectuer un rebase, appliquer un cherry-pick, et finalement publier notre travail sur GitHub.

2. Étape 1 : Initialisation du projet Node.js

La première étape consiste à créer un dossier de projet, initialiser Git et Node.js, puis effectuer un premier commit avec les fichiers de base.

2.1 Commandes exécutées

```
$ mkdir mon-projet-node
$ cd mon-projet-node
$ git init
$ npm init -y
$ echo "# Mon Projet Node" > README.md
$ echo "node_modules/" >> .gitignore
$ echo "console.log('Hello Node');" > index.js
$ git add README.md .gitignore index.js package.json
$ git commit -m "Initialisation du projet Node (README, .gitignore, index.js)"
```

2.2 Résultat obtenu

Le dépôt Git a été initialisé avec succès. Le premier commit contient 4 fichiers : .gitignore, README.md, index.js et package.json.

3. Étape 2 : Gestion des branches

Dans cette étape, nous avons créé une branche dev pour développer une nouvelle fonctionnalité sans affecter la branche principale.

3.1 Commandes exécutées

```
$ git branch dev
$ git checkout dev
```

3.2 Contenu de index.js sur la branche dev

```
function addition(a, b) {
  return a + b;
}
if (require.main === module) {
  console.log("Résultat:", addition(5, 3));
}
module.exports = { addition };
```

```
$ git add index.js
$ git commit -m "Ajout de la fonction addition dans index.js"
```

4. Étape 3 : Fusion (Merge)

Après avoir développé la fonction d'addition sur la branche dev, nous avons fusionné cette branche dans main pour intégrer les modifications.

```
$ git checkout master
$ git merge dev
```

La fusion s'est effectuée en mode Fast-forward, ce qui signifie qu'aucun commit de merge n'a été nécessaire car les historiques étaient linéaires.

5. Étape 4 : Résolution de conflits

Cette étape illustre la gestion des conflits Git. Deux branches ont modifié le même fichier différemment, ce qui a provoqué un conflit lors de la fusion.

5.1 Création du conflit

Sur la branche feature, nous avons ajouté une fonction soustraction. Sur master, nous avons ajouté des commentaires à la fonction addition. Lors de la fusion de feature dans master, Git a détecté un conflit.

```
$ git checkout -b feature
# ... modification de index.js ...
$ git add index.js
$ git commit -m "Ajout de la fonction soustraction"
$ git checkout master
$ git merge feature
CONFLICT (content): Merge conflict in index.js
```

5.2 Résolution du conflit

Nous avons combiné manuellement les deux versions pour conserver à la fois les commentaires et la nouvelle fonction :

```
// addition(a, b) -> retourne la somme de a et b
function addition(a, b) {
  return a + b; // simple addition
}
function soustraction(a, b) {
  return a - b;
}
if (require.main === module) {
  console.log("Résultat addition:", addition(5, 3));
  console.log("Résultat soustraction:", soustraction(5, 3));
}
module.exports = { addition, soustraction };
```

```
$ git add index.js
$ git commit -m "Résolution du conflit entre master et feature"
```

6. Étape 5 : Rebase

Le rebase permet de réécrire l'historique des commits pour obtenir un historique linéaire et plus propre.

```
$ git checkout -b bugfix
# Modification de index.js pour rendre addition plus robuste
$ git add index.js
$ git commit -m "Bugfix: addition robuste (gère les NaN)"
$ git checkout master
$ git rebase bugfix
Successfully rebased and updated refs/heads/master.
```

Le rebase a rejoué les commits de master au-dessus de bugfix, produisant un historique linéaire sans commit de merge supplémentaire.

7. Étape 6 : Cherry-pick

Le cherry-pick permet d'appliquer un commit précis d'une branche sur une autre branche, sans fusionner toute la branche.

```
$ git log --oneline
5bb33d7 (HEAD -> master, bugfix) Bugfix: addition robuste (gère les NaN)
f1e4200 Résolution du conflit entre master et feature
...
$ git checkout feature
$ git cherry-pick 5bb33d7
```

Nous avons appliqué le commit de bugfix sur la branche feature, ce qui lui a permis de bénéficier de la validation NaN sans passer par une fusion complète.

8. Étape 7 : Publication sur GitHub

La dernière étape consiste à pousser le dépôt local vers GitHub pour partager le code et permettre la collaboration.

```
$ git remote add origin https://github.com/ikram02-el/TPs_GenieLogiciel.git
$ git branch -M main
$ git push -u origin main
```

Le dépôt est maintenant accessible publiquement à l'adresse :

https://github.com/ikram02-el/TPs_GenieLogiciel

9. Résumé des commandes Git utilisées

Commande	Description
`git init`	Initialiser un dépôt Git local
`git branch / checkout`	Créer et basculer entre les branches
`git merge`	Fusionner une branche dans une autre
`git rebase`	Réécrire l'historique de commits
`git cherry-pick`	Appliquer un commit précis sur une branche
`git push`	Envoyer les commits vers GitHub

10. Conclusion

Ce TP nous a permis de maîtriser les fonctionnalités essentielles de Git : gestion des branches, fusion, résolution de conflits, rebase et cherry-pick. Ces compétences sont indispensables dans un environnement de développement collaboratif. La publication sur GitHub nous a également initiés aux bonnes pratiques de partage et de collaboration dans les projets logiciels modernes.